



DESENVOLVIMENTO MOBILE REACT NATIVE (AULA 12)

MÓDULO 1 - SEMANA 05



Agenda da Semana

- Revisão POO inicial
- Programação Orientada a Objetos (Avançado)
- Herança em POO
- Importação/Exportação Arquivos
- Projeto integrador
- Exercícios

Continuação da Programação Orientada a Objetos - POO

O problema que originou tudo.

Conceito

- Até agora conseguimos criar um animal usando um objeto.

```
const cachorro = {  
  nome: "Bob",  
  idade: 3,  
  peso: 12,  
};
```

- Mas imagine um pet shop com 100 animais. Teríamos que repetir praticamente o mesmo código várias vezes.

```
const cachorro1 = { ... }  
const cachorro2 = { ... }  
const cachorro3 = { ... }  
const cachorro4 = { ... }
```

Programação Orientada a Objetos - POO

Analogia

Imagine uma fábrica de pelúcias.

- Ela não cria cada urso do zero.
- Ela utiliza um molde.
- Com esse molde, consegue fabricar milhares de pelúcias iguais, mudando apenas algumas características.
- Na Programação Orientada a Objetos acontece exatamente a mesma coisa.

P00

Sem P00:

- Sistema com 50 alunos → 50 variáveis de nome, 50 de idade, 50 de nota, ou até mesmo vários objetos sem conexão real entre si mesmo que dentro de um array.
- Se precisar adicionar um novo campo (ex: e-mail), tem que mudar em dezenas de lugares. O P00 garante que toda aplicação reaja de um mesmo modo.

O que é uma Classe?

Conceito

Uma classe é um molde utilizado para criar objetos.

Ela define:

- Quais informações o objeto terá.
- Quais ações ele poderá executar.

A classe não é um animal.

Ela apenas descreve como um animal deverá ser.

Sintaxe:

```
class Animal {  
  
}
```

Analogia

Classe (Molde)



new
↓



Bob



Thor



Luna



A classe é como um **molde**.



Os animais criados podem possuir informações diferentes.

Criando Objetos com new

Agora utilizamos a classe para criar vários objetos.

```
class Animal {  
  
}  
  
const cachorro = new Animal();  
const gato = new Animal();  
const coelho = new Animal();
```

Na programação: o **new** é como apertar o botão da fábrica para produzir um novo animal.

A palavra new significa:

"Crie um novo objeto baseado nesta classe." e um novo objeto é criado na memória.

Atributos

Conceito

Os atributos representam as **características** do objeto.

Exemplos de Nome, Idade, Peso, Cor Tudo isso são atributos.

```
class Animal {  
    nome;  
    idade;  
    peso;  
}
```


O que é o constructor?

O constructor é um método especial que é executado automaticamente sempre que criamos um novo objeto usando a palavra new.

Sua principal função é inicializar o objeto, ou seja, definir seus atributos com os valores informados no momento da criação. Sem o constructor, o objeto seria criado sem essas informações.

Analogia: Matrícula em um curso

Imagine que você vai se matricular em um curso.

No momento da matrícula, a secretaria solicita informações como:

- Nome
- Idade
- Curso

Esses dados são preenchidos na hora da matrícula, não depois.

O que é o constructor?

Então se fizermos isso:

```
const cachorro = new Animal();
```

Como o JavaScript sabe que ele se chama Bob?

Resposta:

Não sabe. Precisamos informar essas informações.

Quem faz isso? O **Constructor**.

Sintaxe e Exemplo do Constructor

```
class Animal {  
    constructor(nome, idade, peso) {  
        this.nome = nome;  
        this.idade = idade;  
        this.peso = peso;  
    }  
}  
  
// Utilização  
const cachorro = new Animal(  
    "Bob",  
    3,  
    12  
);
```

Mas o que é o this?

O que é o **this**?

O **this** representa o **objeto atual**.

```
class Animal {  
  constructor(nome, idade) {  
    this.nome = nome; // O nome deste objeto  
    this.idade = idade; // A idade deste objeto  
  }  
}
```

Sempre que usamos:

this.algumaCoisa

estamos dizendo:

"A algumaCoisa **deste objeto** aqui"

Exemplo 1: criando um **cachorro**

1 Criamos um objeto:

```
const cachorro = new Animal("Bob", 3);
```

2 O JavaScript executa automaticamente:

```
this.nome = "Bob";  
this.idade = 3;
```

3 O objeto passa a existir assim na memória:



Exemplo 2: criando um **gato**

1 Criamos um objeto:

```
const gato = new Animal("Luna", 2);
```

2 O JavaScript executa automaticamente:

```
this.nome = "Luna";  
this.idade = 2;
```

3 O objeto passa a existir assim na memória:



**PARA
FIXAR**

Leia **this** como:
"o objeto que está sendo criado ou utilizado **agora**".

IMPORTANTE

this não é uma variável.
É uma referência ao objeto atual.

Exercício - Correção

Ticket #482 (Revisão de Code Review):

SISTEMA DE ESTOQUE - LOJA "TECH STORE"

Esse arquivo foi crescendo rápido e cada produto novo virou um objeto solto, copiado e colado do anterior.

Arquivo

Precisamos padronizar a criação de produtos usando CLASSE, para garantir que todo produto novo nasça com os mesmos atributos."

Exercício - Correção

SUA TAREFA:

- 1. Crie uma classe chamada `Produto` com um construtor que recebe: nome, preco, quantidadeEstoque e categoria — e define esses valores como atributos do objeto (this.nome, this.preco, ...).
- 2. Substitua os 5 objetos soltos abaixo (produto1 a produto5) por instâncias da classe, usando `new Produto(...)`.
- 3. NÃO crie métodos dentro da classe (isso é assunto da próxima aula). As funções que já existem no arquivo (calcularValorTotalEstoque, listarProdutosPorCategoria, etc.)
- 4. NÃO se preocupe com herança — ainda não vimos esse conteúdo.
- 5. No final, o `console.log(gerarRelatorioEstoque(produtosEstoque))` precisa continuar funcionando exatamente igual a antes.

Exemplo de Dados dinâmicos

O código a seguir se refere a dados dinâmicos, ou seja que vem da API e nem sempre é o usuário que cria, e sim se baseia para criar outro objeto. No objeto possui herança que veremos em breve sobre isso

[Código Exemplo](#)

Métodos

Os métodos representam as ações ou comportamentos que um objeto pode executar.

Enquanto os atributos armazenam as características do objeto, os métodos definem o que ele é capaz de fazer.

Por exemplo, um objeto Cliente pode possuir métodos como comprar(), vender() e trocar(). Já um objeto Personagem pode executar ações como correr(), pular() e andar().

Em JavaScript, a declaração de métodos é mais simples do que a de funções tradicionais. Dentro de uma classe, não é necessário utilizar a palavra-chave function, pois o JavaScript entende que tudo o que é declarado ali corresponde a um atributo ou a um método da classe.

Métodos

Conceito

- Assim como uma pessoa realiza ações. Um objeto também pode executar ações. Essas ações recebem o nome de Métodos.

Analogia

Um cachorro possui características:

- Nome
- Idade
- Peso

Mas também consegue:

- Comer
- Dormir

Na Programação Orientada a Objetos: características → **atributos** / ações → **métodos**

Exemplo do Método

```
class Animal {  
  constructor(nome, idade, peso) {  
    this.nome = nome;  
    this.idade = idade;  
    this.peso = peso;  
  }  
  //não é necessário utilizar a palavra-chave function, pois o JavaScript entende que tudo o que é  
  declarado ali corresponde a um atributo ou a um método da classe  
  comer() {  
    console.log("O animal está comendo.");  
  }  
  // ou  
  comer() {  
    console.log(`${this.nome} está comendo.`);  
  }  
}
```

class completa

```
class Conta {  
    titular = "";  
    saldo = "";  
    constructor(titular, saldo) {  
        this.titular = titular;  
        this.saldo = saldo;  
    }  
    depositar(valor) {  
        this.saldo += valor;  
    }  
}  
  
const contaAna = new Conta("Ana", 100);  
const contaBruno = new Conta("Bruno", 500);  
contaAna.depositar(50);
```

Exercício - Classe completa

SISTEMA - CONTA BANCÁRIA (NUBANK SIMPLIFICADO)

Precisamos de uma classe `ContaBancaria` para representar as contas dos clientes. Cada conta deve permitir depositar e sacar valores.

SUA TAREFA:

1. Crie a classe `ContaBancaria` com um construtor que recebe titular e saldo.
2. Adicione o método `depositar(valor)`, que soma o valor ao saldo.
3. Adicione o método `sacar(valor)`, que subtrai o valor do saldo.
4. No final, os `console.log` abaixo precisam exibir os resultados esperados.

Herança em POO

O que é Herança?

Conceito

A Herança é um dos pilares da Programação Orientada a Objetos. Ela permite que uma classe reutilize atributos e métodos de outra classe, evitando a duplicação de código. Em vez de recriar tudo do zero, uma nova classe pode aproveitar tudo o que já foi desenvolvido em outra, adicionando apenas as características e comportamentos que a tornam diferente.

Benefícios

- Reaproveitamento de código.
- Menor duplicação.
- Organização da aplicação.
- Facilidade de manutenção.

Herança representa uma relação do tipo "é um". Exemplo:

- Um Cachorro é um Animal.
- Um Gato é um Animal.

HERANÇA EM POO - ANALOGIA

Herança permite que uma classe **reutilize** atributos e métodos de outra classe, adicionando apenas o que é **específico** dela.



IDEIA: Em vez de criar tudo do zero, aproveitamos o que já existe e adicionamos apenas o que é novo.

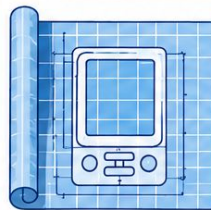
ANALOGIA: FÁBRICA DE ELETRÔNICOS



A fábrica cria um **projeto base** para qualquer eletrônico.

Esse projeto já possui tudo que todo eletrônico precisa ter.

PROJETO BASE: ELETRÔNICO



O que todo eletrônico tem:

- ligar()
- desligar()
- bateria
- tela
- configurar()

RESULTADO



Menos código repetido,
mais **organização** e
manutenção muito
mais fácil!

A partir do projeto base,
criamos eletrônicos específicos.

CLASSE: SMARTPHONE



Herdou do ELETRÔNICO:

- ✓ ligar()
- ✓ desligar()
- ✓ bateria
- ✓ tela
- ✓ configurar()

Adicionou o que é específico:

- tirarFoto()
- fazerLigacao()

CLASSE: NOTEBOOK



Herdou do ELETRÔNICO:

- ✓ ligar()
- ✓ desligar()
- ✓ bateria
- ✓ tela
- ✓ configurar()

Adicionou o que é específico:

- digitar()
- armazenarArquivos()

CLASSE: SMART TV



Herdou do ELETRÔNICO:

- ✓ ligar()
- ✓ desligar()
- ✓ bateria
- ✓ tela
- ✓ configurar()

Adicionou o que é específico:

- conectarWiFi()
- assistirStreaming()

RESUMINDO

- ★ Smartphone, Notebook e Smart TV **SÃO** eletrônicos.
- ★ Eles herdaram tudo que já era comum.
- ★ E cada um adicionou o que os torna **únicos**.

Herança em POO

O problema que a Herança resolve

```
class Animal {  
  constructor(nome, idade) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  comer() {  
    console.log("Comendo...");  
  }  
}
```

```
class Cachorro {  
  constructor(nome, idade) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
  comer() {  
    console.log("Comendo...");  
  }  
  latir() {  
    console.log("Au Au");  
  }  
}
```

Por que copiar o constructor inteiro?

Por que copiar o método comer() de novo?

Porque ainda não usamos **Herança**.

Sintaxe da Herança

```
class Animal {  
  constructor(nome) {  
    this.nome = nome;  
  }  
  
  comer() {  
    console.log(`${this.nome} está comendo.`);  
  }  
}  
  
class Cachorro extends Animal {}
```

extends significa: Esta classe herda tudo da classe Animal.

Como utilizar a herança

Já sabemos sobre a sintaxe da herança mas como utilizar de fato? Vamos ter que falar de um novo problema

```
class Animal {  
  constructor(nome, idade) {  
    this.nome = nome;  
    this.idade = idade;  
  }  
}  
  
class Cachorro extends Animal {  
  constructor(nome, idade, raca) {  
    this.raca = raca;  
  }  
}  
  
const dog = new Cachorro("Bob", 3, "Labrador");
```

Quem vai definir o nome e a idade? Ninguém. O constructor da classe Animal nunca foi executado.

O que é o `super()`

Conceito

A função **`super()`** executa o constructor da classe pai.

Isso permite que a classe filha aproveite toda a inicialização já existente, evitando repetir código.

Depois que o constructor da classe pai termina sua execução, o constructor da classe filha continua normalmente, podendo adicionar apenas suas próprias informações.

Sempre que uma classe filha possui um constructor, ela deve chamar `super()` antes de utilizar `this`.

SUPER() - A ANALOGIA

O `super()` chama o constructor da classe `pai` antes de executar o que é específico da classe `filha`.



ANALOGIA: CADASTRO DE FUNCIONÁRIOS

1 RH (CLASSE PAI)

Cadastro básico (comum)



DADOS CADASTRADOS PELO RH:

- ✓ Nome
- ✓ CPF
- ✓ Data de nascimento
- ✓ Endereço

2 SETOR ESPECÍFICO (CLASSE FILHA)

Cadastro específico



DADOS ADICIONADOS PELO SETOR:

- ✓ Linguagem favorita
- ✓ Senioridade
- ✓ Pretensão salarial

3 FUNCIONÁRIO CADASTRADO

Dados completos!



DADOS FINAIS DO FUNCIONÁRIO:

- ✓ Nome
- ✓ CPF
- ✓ Data de nascimento
- ✓ Endereço
- ✓ Linguagem favorita
- ✓ Senioridade
- ✓ Pretensão salarial



RESUMO DA ANALOGIA

O `super()` é como o RH: ele executa primeiro tudo que é comum (classe `pai`). Depois disso, a classe `filha` continua e adiciona apenas o que é específico dela.

Código final de Herança

```
class Animal {  
    constructor(nome, idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
}  
  
class Cachorro extends Animal {  
    constructor(nome, idade, raca) {  
        super(nome, idade);  
        this.raca = raca;  
    }  
}  
  
const cachorro = new Cachorro('Rex', 5, 'Labrador');  
console.log(cachorro);
```

Exemplo guiado

Este exemplo mostra como funciona a herança em JavaScript.

A classe Pessoa é a base, e as classes Aluno e Professor herdam suas características, reutilizando código e organizando melhor o programa. Na prática, isso ajuda a criar programas mais limpos e fáceis de entender.

Prática ao vivo:

Vamos aplicar esse conceito em uma atividade prática, passo a passo, para ver a herança funcionando na prática.

[Código](#)

Exercício - Consolidação

Ticket #513 (Sprint Review):

SISTEMA DE CADASTRO - APP DE DELIVERY (POO)

Sistema: Cadastro de usuários

Prioridade: **Alta**

Relato: "Precisamos estruturar o cadastro da plataforma. Cliente e Entregador compartilham dados básicos, mas cada um tem regras de negócio próprias (desconto, ganho do dia). Além disso, o time de suporte quer ver os dados completos de cada tipo sem duplicar o método que já exibe nome/email/telefone."

Exercício - Consolidação

SUA TAREFA:

1. Classe Usuario (pai)

- constructor(nome, email, telefone)
- telefone " " → "Não informado"
- exibirDados(): imprime os 3 campos

2. Classe Cliente extends Usuario

- Atributos: endereco, limiteCredito
- calcularDesconto(valorCompra): 10% se > 100, senão 0
- exibirDadosCliente(): exibirDados() + endereco + limiteCredito

3. Classe Entregador extends Usuario

- Atributos: veiculo, placa, valorPorEntrega
- calcularGanhoDia(qtdEntregas): $qtdEntregas * valorPorEntrega$
- exibirDadosEntregador(): exibirDados() + veiculo + placa + ganho (5 entregas)



DESENVOLVIMENTO MOBILE REACT NATIVE (AULA 13)

MÓDULO 1 - SEMANA 05



Importação de Arquivos (require)

require x import

Qual a diferença?

Ambos servem para importar módulos, mas pertencem a sistemas diferentes.

require	import
CommonJS	ES Modules (ESM)
Node.js tradicional	Padrão oficial do JavaScript

Resumo: Ambos importam código, porém utilizam padrões diferentes.

O que é require?

CommonJS

Muito utilizado em versões mais antigas do [Node.js](#).

```
const fs = require("fs");
```

Características

- Carrega módulos durante a execução.
- Pode ser usado dentro de funções, loops e condicionais.
- Utiliza `module.exports`.

Exemplo com bibliotecas que usam CommonJS

```
const fs = require('fs')           // pacote para ler/escrever arquivos
const path = require('path')       // pacote para trabalhar com caminhos
const input = require('prompt-sync')() // pacote para ler dados do terminal
```

Exemplo de require local

Arquivo [matematica.js](#)

```
function somar(a, b) {  
  return a + b;  
}  
  
module.exports = somar;
```

Arquivo main.js

```
const somar = require("./matematica");  
  
console.log(somar(10, 5));
```

Importando / Exportando Classes

Com os conceitos de importação e exportação apresentados, já podemos aplicá-los na criação de classes.

Em um sistema orientado a objetos, cada classe representa uma entidade do domínio da aplicação. Por esse motivo, é uma boa prática organizar cada classe em seu próprio arquivo. Por exemplo:

- Usuario.js
- Animal.js
- Veiculo.js

Essa organização torna o código mais modular, facilita a manutenção e permite reutilizar as classes em diferentes partes do sistema por meio de export e import.

Nos próximos slides veremos a forma recomendada de realizar essas exportações e importações na prática.

Aplicando Export de Classe

```
// Arquivo personagem.js
class Personagem {
  constructor(nome, caracteristicaPrincipal) {
    this.nome = nome
    this.caracteristicaPrincipal = caracteristicaPrincipal
  }

  correr() {
    console.log(`${this.nome} está correndo`)
  }
}

module.exports = Personagem
```

Aplicando import de classe

```
const Personagem = require("./class");  
  
const personagem1 = new Personagem("Guerreiro Alfa", "força");  
personagem1.correr();
```

Exercício - Consolidação (Parte 2)

4. NÃO sobrescreva `exibirDados()` em nenhuma classe filha (assunto da próxima aula) — ele deve continuar existindo só no pai.
5. Crie um array `usuariosCadastrados`` com pelo menos 2 clientes e 2 entregadores.
6. Usando um `forEach()`, percorra o array chamando `exibirDados()` em cada item. Repare: o mesmo método funciona para qualquer tipo dentro do array, mesmo sem saber de antemão se é `Cliente` ou `Entregador`.
7. Fora do loop, chame `exibirDadosCliente()` de um cliente específico e `exibirDadosEntregador()` de um entregador específico.

Observação: Crie um arquivo separado para as classes

```
module.exports = { Usuario, Cliente, Entregador };
```

DESAFIO EXTRA (opcional):

Calcule e exiba o desconto de um cliente para uma compra de R\$250 e o ganho do dia de um entregador com 8 entregas.

Sobrescrevendo métodos

Sobrescrita de Métodos

Conceito

A sobrescrita de métodos (Override) acontece quando uma classe filha cria um método com o mesmo nome de um método existente na classe pai.

Ao fazer isso, o método da classe filha substitui o comportamento herdado, permitindo personalizar sua execução.

A classe continua herdando todos os demais atributos e métodos da classe pai, alterando apenas aquele comportamento específico.

Quando utilizar?

- Quando o comportamento padrão não atende às necessidades da classe filha.
- Quando diferentes objetos executam a mesma ação de maneiras diferentes.

Analogia da sobrescrita

Analogia: Manual da Empresa

Imagine uma empresa que possui um manual para todos os funcionários. Todo funcionário deve cumprir a seguinte regra:

Início do expediente:
→ Registrar ponto na entrada.

Agora imagine que existe um setor chamado Home Office.

Os colaboradores deste setor não registram o ponto na catraca da empresa. Eles registram o ponto pelo sistema online. O nome da ação continua sendo: **Registrar ponto**

Mas a forma como ela acontece mudou.

Na Programação Orientada a Objetos acontece exatamente a mesma coisa. A classe filha mantém o mesmo método, porém altera sua implementação.

Antes da sobrescrita

```
class Funcionario {  
  
    registrarPonto() {  
        console.log("Registrando ponto na catraca.");  
    }  
  
}  
  
class Desenvolvedor extends Funcionario {  
  
}  
  
const dev = new Desenvolvedor();  
dev.registrarPonto();
```

Resultado: Registrando ponto na catraca

Quem executou esse método? A classe pai.

Sobrescrevendo

```
class Desenvolvedor extends Funcionario {  
  
    registrarPonto () {  
        console.log("Registrando ponto pelo sistema remoto. ");  
    }  
  
}  
  
const dev = new Desenvolvedor();  
  
dev.registrarPonto();
```

Resultado Registrando ponto pelo sistema remoto.

Agora quem foi executado? O método da classe filha.

Exercício – Sobrescrita de Métodos (Override)

Contexto

Uma empresa está desenvolvendo um sistema para enviar notificações aos seus clientes.

Inicialmente, o sistema possuía apenas uma classe chamada Notificação, responsável por enviar qualquer tipo de mensagem. Porém, com o crescimento da empresa, surgiram diferentes canais de comunicação, como e-mail, SMS, notificações push e WhatsApp.

Cada canal envia a mensagem de uma forma diferente, mas todos possuem a mesma responsabilidade: enviar uma notificação.

Seu desafio será utilizar herança e sobrescrita de métodos (override) para representar esse cenário.

Exercício – Sobreescrita de Métodos (Override)

Requisitos

1. Crie uma classe chamada Notificacao

Essa classe deve possuir:

- Um atributo para armazenar o destinatário.
- Um método chamado:
 enviar(mensagem)

O método da classe base deve apenas exibir: Enviando notificação...

2. Crie as seguintes classes que herdam de Notificacao

- Email
- SMS
- PushNotification
- WhatsApp

Cada uma deve sobrescrever o método: enviar(mensagem)

Exercício – Sobrescrita de Métodos (Override)

Cada classe deverá apresentar uma mensagem diferente.

Exemplo esperado:

```
Enviando e-mail para João  
Mensagem: Promoção disponível!
```

```
Enviando SMS para João  
Mensagem: Promoção disponível!
```

```
Enviando Push Notification para João  
Mensagem: Promoção disponível!
```

```
Enviando Push Notification para João  
Mensagem: Promoção disponível!
```

Exercício – Sobreescrita de Métodos (Override)

3. Crie um objeto de cada tipo.

Exemplo:

```
const email = new Email("João");  
const sms = new SMS("Maria");  
const push = new PushNotification("Carlos");  
const whatsapp = new WhatsApp("Fernanda");
```

4. Armazene todos em um único array

5. Percorra o array

Utilize um `forEach()` para percorrer o array e envie a mesma mensagem para todos.

Funções Avançadas e Closures

Funções dentro de funções

```
function externa() {  
  
    function interna() {  
        console.log("Estou dentro");  
    }  
  
    interna();  
}  
  
externa();
```

A função interna existe fora da externa? Erro. Porque ela nasceu apenas naquele escopo.

Escopo

Variáveis existem apenas onde foram criadas.

```
function teste() {  
  
    let nome = "Matheus";  
  
    console.log(nome);  
  
}  
  
teste();  
  
console.log(nome);
```

Analogia: Cada função é como uma sala. Quem está dentro da sala consegue ver tudo que está nela. Quem está do lado de fora não consegue enxergar.

Closure

Uma closure é quando uma função "lembra" das variáveis do lugar onde ela foi criada, mesmo depois que esse lugar já terminou de executar.

É como um funcionário que sai da empresa, mas ainda lembra da senha do Wi-Fi:

- A empresa (função externa) já fechou o expediente
- O funcionário (função interna) ainda carrega a informação consigo
- Ele consegue usar a senha quando quiser

Definição técnica

Uma closure é formada por dois elementos:

- A própria função.
- A referência ao ambiente léxico (lexical environment) onde essa função foi criada.

Esse ambiente contém todas as variáveis que estavam disponíveis naquele momento.

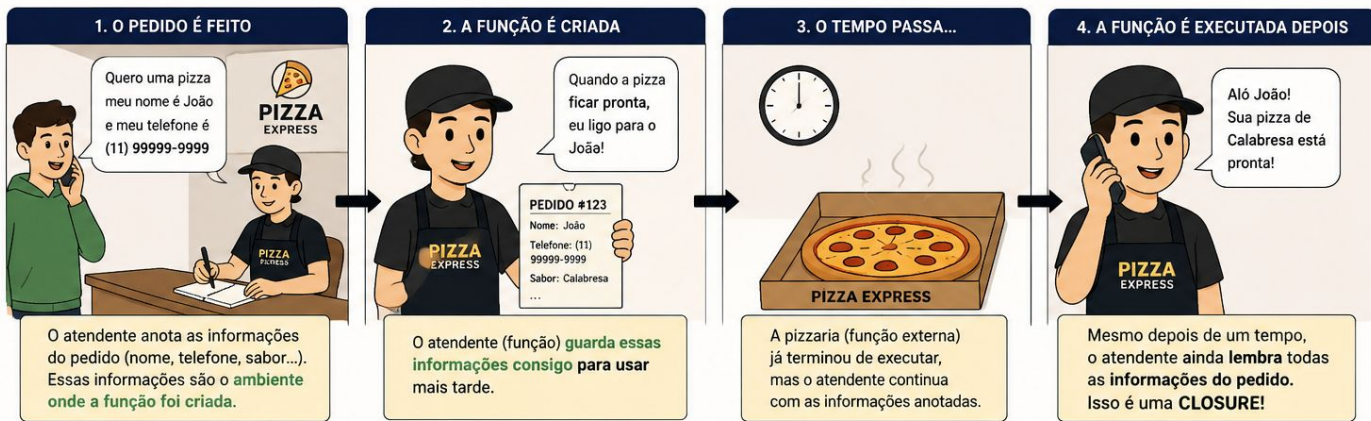
Closure

Casos de uso

- Criar `contadores` que guardam o valor entre chamadas
- Criar `funções fábrica` — uma função que gera outras funções personalizadas
- Event listeners e callbacks no navegador (veremos mais adiante)]
- `setTimeout` (exemplo mais utilizado)

CLOSURE: A FUNÇÃO QUE TEM MEMÓRIA

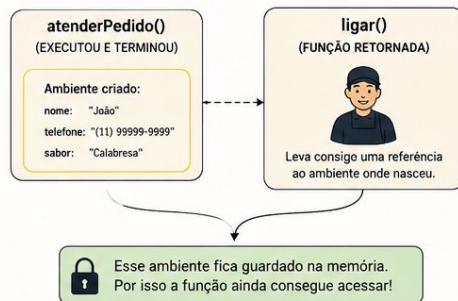
Closures permitem que uma função “lembre” das informações do lugar onde foi criada, mesmo depois que esse lugar já terminou sua execução.



TRADUZINDO PARA O CÓDIGO

```
function atenderPedido(nome, telefone, sabor) {  
  // Essas variáveis são o ambiente  
  return function ligar() {  
    console.log('Alô ${nome}!');  
    console.log('Sua pizza de ${sabor} +  
      ' está pronta!');  
    console.log('Telefone: ${telefone}');  
  };  
}  
  
const ligarParaJoao = atenderPedido(  
  "João", "(11) 99999-9999", "Calabresa"  
);  
  
// ...tempo passa...  
ligarParaJoao(); // A função ainda lembra de tudo!
```

O QUE ACONTECE?



ONDE VOCÊ USA ISSO SEM PERCEBER?

- ✓ setTimeout / setInterval
- ✓ Event Listeners (cliques, teclas...)
- ✓ Callbacks
- ✓ Promises
- ✓ Async / Await
- ✓ React (useState, useEffect, etc.)
- ✓ E MUITO MAIS!



★ RESUMO QUE VIRA A CHAVE

A função nasce em um lugar, guarda as informações daquele lugar e leva consigo para usar quando precisar. Essa é a **mágica** (e o poder) das **Closures**!

Sintaxe closure

```
function funcaoExterna(parametro) {  
  let variavel = "valor";  
  
  return function funcaoInterna() {  
    // aqui dentro consigo acessar:  
    // - parametro (da função externa)  
    // - variavel (da função externa)  
  };  
}  
  
const minhaFuncao = funcaoExterna("algo");  
minhaFuncao(); // executa a função interna, que ainda "lembra" de tudo
```

Exemplo de Closure

```
function contador() {  
  let numero = 0;  
  
  return function () {  
    numero++;  
    console.log(numero);  
  };  
}  
  
const contar = contador();  
  
contar();  
contar();  
contar();
```

Como a variável ainda existe se a função contador terminou? Porque a função retornada "lembra" do ambiente onde nasceu. Isso é uma Closure.

O que não é uma closure?

Ter uma função dentro de outra não significa, por si só, que existe uma closure.

```
function externa() {  
  function interna() {  
    console.log("Olá");  
  }  
  interna();  
}
```

Aqui existe apenas uma função aninhada. A closure só se torna relevante quando a função interna continua existindo e acessando variáveis do escopo externo após esse escopo ter sido encerrado.

"Closure é a capacidade de uma função carregar consigo o ambiente onde foi criada."

Exemplo de Closures - Máquina de Café

Você configura: sem açúcar, leite médio. Toda vez que aperta o botão, sai o café do seu jeito — ela guardou sua configuração. Em código seria:

```
function configurarCafe(acucar, leite) {  
  return function () {  
    console.log(`Cafe ${acucar} acucar e leite ${leite}`);  
  };  
}  
  
const meuCafe = configurarCafe("sem", "medio");  
meuCafe(); // Cafe com sem acucar e leite medio  
meuCafe(); // sempre igual — lembra da configuracao!
```

Exercício — Sistema de Curtidas

Cenário

Você foi contratado para desenvolver um sistema simples semelhante ao de uma rede social.

O sistema deverá lembrar quantas curtidas um post possui.

Todas as informações devem permanecer armazenadas através de uma Closure.

```
===== POST =====
```

```
1 - Curtir 👍
```

```
2 - Descurtir 👎
```

```
3 - Exibir curtidas ❤️
```

```
4 - Sair
```

```
Escolha:
```

O menu deverá permanecer aberto até que o usuário escolha a opção 4.

Exercício — Sistema de Curtidas

Regras

Crie uma função chamada: `criarSistemaCurtidas()`

- Ela deverá possuir internamente apenas uma variável: `let curtidas = 0;`
- Essa variável não poderá ser global.

A função deverá retornar uma única função responsável por controlar todas as ações do sistema.

Essa função receberá como parâmetro a ação que deverá ser executada. Exemplo:

```
const sistemaCurtidas = criarSistemaCurtidas();  
sistemaCurtidas("curtir");  
sistemaCurtidas("descurtir");  
sistemaCurtidas("consultar");
```

Dicas para montar o menu: Utilize `prompt-sync` para montar o menu, Utilize `while/do while`. Utilize `switch` para opção

Callback

O que é um Callback?

Conceito

Callback é uma função passada como argumento para outra função, para ser executada em um momento específico.

Ou, de forma ainda mais simples: É uma função que fica "esperando ser chamada".

É como deixar um bilhete com instruções para alguém:

- Você entrega o bilhete (a função) para outra pessoa (outra função)
- Essa pessoa decide quando vai ler e seguir as instruções
- Você não precisa ficar esperando, ela executa quando for a hora

Callback

Imagine que você pediu uma pizza.

Você diz para a pizzeria:

- > *"Quando a pizza ficar pronta, me liga."*

Você não sabe quando isso vai acontecer. Você apenas deixou uma ação para ser executada depois.

Essa ação é o callback.

Casos de uso

Casos de uso

- `forEach`, `map`, `filter` — vocês já usaram callbacks sem saber!
- `setTimeout` e `setInterval` — executar algo depois de X segundos
- Leitura de arquivos — "quando terminar de ler, faça isso"
- Cliques em botões — "quando o usuário clicar, execute esta função"

Sintaxe callback

```
function executarDepois(callback) {  
  // faz alguma coisa...  
  callback(); // executa a função que foi passada  
}  
  
// Chamo a funcao executarDepois e passo como parametro uma outra funcao  
executarDepois(function () {  
  console.log("Fui executada!");  
});
```

Sintaxe Callback

- `callback` — É só um parâmetro normal, mas em vez de receber um número ou texto, recebe uma função inteira.
- `callback()` — Usamos `()` para executar a função que foi passada. Sem os parênteses, ela é só uma referência — com os parênteses, ela roda.
- `function () { ... }` — A função que passamos como argumento. Pode ser anônima (sem nome) ou nomeada.
- `Callback com parâmetros` — O callback pode receber dados — quem chama o callback decide o que passar.

Exemplo callback

```
// Callback com parâmetro
function buscarDado(callback) {
  const dado = "Resultado da busca"
  callback(dado) // passa o dado para o callback
}

buscarDado(function (resultado) {
  console.log(resultado) // "Resultado da busca"
})

// Ou

function cumprimentar(nome) {
  console.log(`Olá ${nome}`);
}

function executar(callback) {
  callback("Matheus");
}

executar(cumprimentar);
```

Outros exemplos de Callback

Onde usamos Callback sem perceber?

Todos esses utilizam callbacks:

```
setTimeout(() => {  
  
}, 1000);
```

```
array.forEach((item) => {  
  
});
```

```
button.addEventListener("click", () => {  
  
});
```

CALLBACK

O que é?

Callback é quando passamos uma **função** como parâmetro para outra função, e essa função é **executada depois**, em algum momento oportuno.



ANALOGIA: PIZZARIA



A **pizzaria** = função que recebe o callback
O **aviso** quando pronto = **callback**



ONDE ISSO É USADO?



forEach
(percorrer listas)



Eventos
(cliques, teclas)



setTimeout
(esperar algo)



Requisições
(buscar dados)



IDEIA PRINCIPAL

Eu (você) falo para outra função:
"Quando terminar o que você vai fazer,
pode **executar esta função aqui por mim**."



EXEMPLO NO CÓDIGO

```
// Função que recebe um callback
function prepararPedido(cliente, quandoPronto) {
  console.log(`Preparando pedido de ${cliente}...`);

  // ... tempo passando (preparando) ...
  // Quando o pedido fica pronto, executa o callback
  quandoPronto(cliente);
}

// Função de callback
function avisarCliente(cliente) {
  console.log(`Pedido de ${cliente} está pronto!`);
}

// Passando a função avisarCliente como callback
prepararPedido("Matheus", avisarCliente);
```

SAÍDA NO CONSOLE:

```
Preparando pedido de Matheus...
Pedido de Matheus está pronto!
```



A função **avisarCliente**
foi executada lá dentro
de **prepararPedido**.



RESUMO

Callback é uma função passada como parâmetro para outra função, que será chamada (executada) **depois**, quando algo acontecer.

Outros exemplos de Callback

Onde usamos Callback sem perceber?

Todos esses utilizam callbacks:

```
setTimeout(() => {  
  
}, 1000);
```

```
array.forEach((item) => {  
  
});
```

```
button.addEventListener("click", () => {  
  
});
```

Exercício Callback

CHAMADO DO SUPORTE

Sistema: iFoodGo - Módulo de Notificações

Prioridade: Média

Relato: "Precisamos percorrer os pedidos e notificar o cliente, mas cada canal (App, Painel Interno) faz algo diferente com a notificação. Crie UMA função que percorra os pedidos e, para cada um, chame uma função (callback) passada por quem chamou, entregando a mensagem pronta."

Exercício - Callback

1) Implemente processarPedidos(pedidos, callback):

- Percorra o array "pedidos" com forEach
- Para cada pedido, monte uma mensagem de acordo com o status:
 - "entregue" -> "Pedido #ID entregue para CLIENTE!"
 - "cancelado" -> "Pedido #ID foi cancelado."
 - qualquer outro -> "Pedido #ID está em andamento (STATUS)."
- Chame callback(mensagem) passando a mensagem montada (é essa chamada que entrega a mensagem pra fora da função)

2) Chame processarPedidos DUAS vezes, passando um callback diferente em cada chamada:

- um callback que só faz console.log(mensagem) com 📱 na frente
- um callback que não loga nada, só soma 1 na variável totalPedidos

Código base: [Código](#)

INFORMAÇÕES DO TEMPLATE

- Título da Aula:
 - Fonte: Roboto
 - Formato: Maiúsculo
 - Tamanho: 32
 - Cor: Branco
- Sub-título da Aula:
 - Fonte: Roboto
 - Formato: Maiúsculo
 - Tamanho: 16
 - Cor: Verde
- Título do Slide:
 - Fonte: Roboto
 - Formato: Maiúsculo
 - Tamanho: 24
 - Cores: Verde
- Parágrafos:
 - Fonte: Roboto
 - Tamanho: 12 a 18
 - Cores: Branco (Modo Escuro) ou Preto (Modo Claro)
- Marcadores de tópicos:
 - Formatos: Símbolos ou Alfanuméricos
 - Cor: Verde

EXEMPLO COM TEXTO (MODO CLARO)

O NOMEPRODUTO, é uma formação imersiva do LAB 365, alinhada com as mais recentes tecnologias de mercado, com o objetivo de acelerar a carreira de NOMEAREA.

EXEMPLO COM TEXTO (MODO ESCURO)

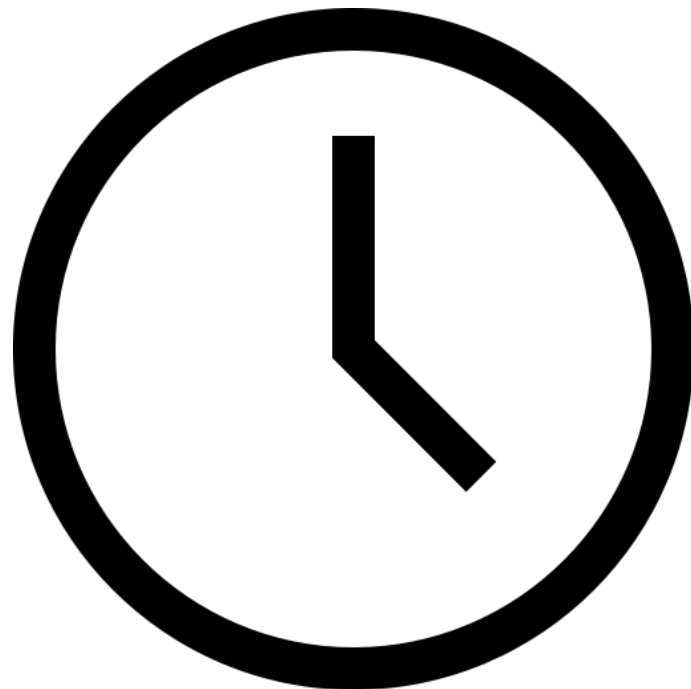
O NOMEPRODUTO, é uma formação imersiva do LAB 365, alinhada com as mais recentes tecnologias de mercado, com o objetivo de acelerar a carreira de NOMEAREA.

INTERVALO!

Finalizamos o nosso primeiro período de hoje. Que tal descansar um pouco?!

Nos vemos em 15 minutos.

Início: 20:30
Retorno: 20:45



AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique [aqui](#) ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.